PROJECT SPECIFICATION

LoopZero

A self-verifying autonomous coding harness

One raw, unseen product idea in. One tested, running, persistent web application out — with the harness, not the model, owning every claim about whether it works.

Team	LoopZero
Members	Hossam Elshahaby · Paul Grönborg · Ali Sina Mohamed Akif · Shivam Gupta
Event	AgentCofounder Hackathon · Stockholm AI
Track	Contracts
Agent framework	Pi (pinned version)
Model / provider	Qwen3 27B FP8 on Berget AI (Sweden)
Deliverables	agentcofounder-optimized.zip · OPTIMIZATION_REPORT.md

1. Problem statement

Two problems, stacked.

1.1 The founder problem

"I have a great idea, but no one to build it with." Hiring a technical cofounder is slow, agencies are expensive, and the momentum dies long before anything exists to put in front of a user.

1.2 The agent problem

Pointing a coding agent at a vague, one-sentence idea produces drift:

- half-finished features that stop at the first interesting screen;
- no tests, or a suite of skipped tests that reports green;
- no persistence — reload the page and the work is gone;
- code that is not maintainable or extensible afterwards;
- and, critically, **no verification**: a demo that only runs on the author's laptop proves nothing to a judge or an investor.

LoopZero targets the second problem so that the first becomes tractable.

2. Objective and scope

Objective. Given an unseen raw startup idea, reliably produce a runnable, tested, persistent, maintainable proof of concept while maximising the evaluation score and minimising weighted model-token cost.

In scope. The harness: prompting, orchestration, verification, repair, telemetry, and the reusable application template.

Out of scope. Building any single vertical application. LoopZero is domain-neutral by construction — no idea-specific logic exists anywhere in the repository.

Hard constraints. Pi remains the coding framework; Qwen on Berget AI remains the model and provider; the public contract and the result schema are unmodified; protected paths are not weakened; no new runtime dependencies.

3. System architecture

```
raw idea (one sentence, unseen)
├─ Phase 1 understand + structure → compact idea spec
├─ Phase 2 build → Pi agent on Qwen writes the app
│ on top of app-template
├─ Phase 3 verify (runner-owned) → vitest → production build →
│ dev server :3000 → HTTP probe → cleanup
├─ Phase 4 repair (bounded, ≤1) → failing checks + log tails only
│ → re-verify into verify-N/
└─ Phase 5 report → trace.jsonl · summary.md · result.json
```

The model never reports its own success. Every fact written into result.json — tests passed, build clean, server reachable, tokens consumed — is measured by the runner from process exit codes, JSON reports and HTTP responses.

3.1 Components

Module	Responsibility
src/run-challenge.ts	Phase pipeline, per-phase timeouts, prompt assembly, orchestration
src/verify-app.ts	Deterministic verification: tests → build → startup → HTTP probe
src/repair.ts	Single bounded repair call built from failing checks + log tails
src/usage.ts	Exact usage aggregation across all Pi event streams (mergeUsageSummaries)
src/trace.ts	Append-only per-phase trace.jsonl
src/result.ts / validate-result.ts	Emit and schema-validate result.json (contract, unmodified)
src/port-owner.ts / process-tree.ts	Port ownership checks and clean process-tree teardown
src/prepare-output.ts	Workspace preparation and template copy
solution/system-prompt.md	Role, method and cache-friendly stable prefix
solution/skills/mvp-builder	Domain-shape playbook, inlined into the prompt
app-template/	Reusable toolkit + accessible UI kit + passing test baseline

4. Deterministic verification contract

A run is accepted only when every condition below holds. There is no partial pass.

Check	Pass condition
Unit tests	vitest JSON report: success = true, numTotalTests > 0, all passed, zero failed, zero pending, zero todo, zero skipped
Production build	build command exits 0
Server startup	dev server binds port 3000 and the port is owned by the spawned process tree
HTTP probe	the served application answers a real HTTP request
Persistence	state survives a reload — enforced by the template's storage layer and the app's own tests
Teardown	the whole process tree is terminated; no orphan holds the port
Result	result.json validates against the unmodified contract schema

A suite consisting only of skipped or todo tests is treated as a failure, not a pass. This closes the most common way an agent can appear green while proving nothing.

4.1 Bounded repair

When any check fails, the runner issues exactly one focused follow-up call containing the failing check names and the tail of each failing log — not the full transcript. Re-verification writes to a fresh verify-N/ directory so the original evidence is preserved. The loop is capped and cannot become a token sink.

5. Token-efficiency strategy

Efficiency is scored as:

```
weighted cost = input_tokens + output_tokens × 3 + cache_read_tokens × 0.1
```

Output is charged triple, so the cheapest line of code is the one the model never writes. Two levers follow directly.

5.1 Ship the boilerplate in the template

Toolkit module	What the generated app gets for free
storage.ts	Typed localStorage with versioning and corrupt-data recovery
repository.ts	CRUD repository over storage
validation.ts	Field validators and form-level error shaping
dates.ts	ISO parsing, relative formatting, comparison
collection.ts	Filter, sort, group and search over record sets
state-machine.ts	Declarative lifecycles (draft → active → done)
id.ts	Collision-resistant id generation
useCollection.ts	React hook binding a repository to component state
src/ui/index.tsx	Accessible kit: form, table, dialog, toast, empty/loading/error, shell
src/test/setup.ts, helpers.ts	Vitest environment and render helpers

All modules are domain-neutral; 16 template tests give every generated app a passing baseline from the first commit.

5.2 Cache-aware prompt hierarchy

- Stable role and method text first, volatile idea last — the invariant prefix is cache-eligible at 0.1× weight.
- The mvp-builder skill is inlined into the prompt instead of being read from a file path, removing an entire read turn from every run.
- The skill is organised around domain shapes — workflow, planner, log, comparison, checklist, register — so an unseen idea maps onto a known plan instead of being improvised.

6. Measured results

Metric	Value
Model	openai/Qwen/Qwen3.8-27B-FP8 on Berget AI
Total model calls	19
Total execution time	796.3 s (~13.2 minutes)
Pass rate	100% — 0 repair attempts required
Total token consumption	491,343
Input tokens	468,472 (95.3% of total)
Output tokens	22,871 (4.7% of total)
Weighted cost score	537,085 (input + output × 3)

Output tokens at 4.7% of the total is the headline figure: the toolkit absorbed the boilerplate, so the expensive channel stayed nearly empty.

6.1 Demo run

Unseen idea: "I run a small pottery studio and I keep running out of glazes and clay at the worst possible moment."

Result: a running studio-supplies application on port 3000 — add a supply, filter by type (glaze / clay / tools), adjust quantities inline, edit, delete, and a "Running low" panel driven by a configurable threshold. Refreshing the browser keeps every row, because persistence is part of the verification contract rather than an afterthought.

7. Hidden-idea robustness suite

Six synthetic ideas, one per domain shape, deliberately unlike the public example so the harness cannot overfit. Each runs via --idea-file.

Benchmark idea	Domain shape
01-workflow-repair-shop	Workflow / state machine
02-planner-plant-care	Recurring planner
03-log-freelance-hours	Time log with aggregation
04-comparison-energy-bills	Comparison / trend
05-checklist-rental-inspection	Checklist with completion state
06-register-lending-library	Register with lending lifecycle

8. Operations

```
npm ci --ignore-scripts
npm run check # lint + typecheck + 47 harness + 16 template tests
npm run challenge -- --prepare-only
npm run challenge -- --idea-file benchmark/ideas/01-workflow-repair-shop.txt
```

Credentials are read from the environment only; nothing is written into the repository or the distributed ZIP. Artifacts land in output/<timestamp>/: the generated application, verify-*/ logs, trace.jsonl, summary.md and result.json.

9. Contract compliance

Deliberately untouched:

- contract-public/** and the result.json schema
- src/result.ts and src/validate-result.ts
- the semantics of src/verify-app.ts — ordering and pass conditions
- solution/extensions/protected-paths.ts — protections are not weakened
- the pinned Pi agent version and both lockfiles

No new runtime dependencies were introduced. The runner — never the model — owns test, build, startup, token and success facts.

10. Impact

Audience	What changes
Founders	A real, testable artifact in minutes instead of a slide deck.
Engineering teams	Verification moves into the harness; agent output is accepted only when tests, build and startup all pass.
Agent ecosystem	An open, auditable contract — trace.jsonl, summary.md and result.json make every run replayable and comparable.
Nordic AI stack	Runs entirely on Berget AI with Qwen — sovereign European compute, no US hyperscaler in the loop.